```
Output:
string(5) "Alice"
string(3) "Bob"
string(3) "Bob"
string(5) "Alice"
```

While it's true that array access can be flexible, keeping syntax clean and simple is recommended. Common practice is to assign the result of an expression or function call to a variable first and then use that as the key.

Associative Arrays

When array elements are created, they are automatically given index values that can be used as keys to access them. It is also possible (and often desirable) to specify your own keys. Here's how it is done:

```
$myArr = [
    'lang' => 'PHP',
    'ver' => '8.4'
];

print_r($myArr);
print "Version of {$myArr['lang']} is {$myArr['ver']}\n";

Output:
Array
(
    [lang] => PHP
    [ver] => 8.4
)
Version of PHP is 8.4
```

Keys are associated with elements using the double-arrow operator =>. Legal values for keys are limited to decimal integers and strings (or expressions that evaluate to them). Type coercion rules apply to keys, so take care with types. (Don't try to use floats as keys, e.g., it will work with PHP 8, but it is deprecated due to precision loss). The array elements are accessed using the keys, which in this case are the strings "lang" and "ver".